Vision-5: Röstvägledning

Vad som byggdes, varför, hur det är testat och vad som återstår, 3 november 2026

Byggt på en dag efter mätserierna 17–29/10 (PR #16). Koden ligger i voice_guidance/, reglerna och protokollet i docs/ROSTVAGLEDNING.md. **Inget är körd på Pi:n än.** Allt är testat med låtsaskamera, äkta streckkoder skapade av zxing-cpp och påhittade loggar.

Sammanfattning

Vad: en motor som tittar på varje avläsning och säger åt användaren vad hen ska göra för att få en kod läst säkert (”lite närmare”, ”lite åt vänster”, ”håll stilla”), och bekräftar varan med namn när läsningen är säker. **Varför:** mätningarna visade att koden läses säkert när den är stor nog, ungefär rak och mitt i bilden. En seende ser det själv. En blind användare behöver få veta det. **Hur:** ren logik utan kamera och ljud, uppdelad i delar med ett ansvar var, så att den går att testa, spela upp mot gamla loggar och portera till C#.

1. Reglerna och var de kommer ifrån

Läge	Besked	Mätning som ligger bakom
Två olika koder från samma ställe inom 3 s	Håll stilla	Kaffefiltret 29/10: rätt och fel kod om vartannat, 22 felläsningar
Samma vara läst säkert två gånger	Varans namn, eller ”Okänd vara”	Felläsningar kan ha giltig kontrollsiffra. Namn bara för varor i listan
Koden syns men är osäker, ligger i kanten	Riktning	Autofokus följer bildens mitt: kod längst ner 0 %, i mitten 100 % (20/10)
Koden syns men är osäker, ligger i mitten	Lite närmare	Nära pixelgränsen är även rätta läsningar 1 px höga (27/10): tystnad hade varit att ge upp
Allt annat	Tystnad	Pratar hellre för lite än för mycket

Takt: minst 1,8 s mellan beskeden, samma besked inte oftare än var 3:e s. Bekräftelsen går före. Alla trösklar ligger i GuidanceConfig och är startvärden från mätningarna.

2. Hur det är uppbyggt

Del	Ansvar	Vet inget om
model.py	CodeRead, FrameObservation, GuidanceAction. Enda stället som läser loggradens format	ljud, kamera
engine.py	GuidanceEngine: vad ska sägas nu? Tiden följer med bildrutan	ljud, kamera, klocka
catalog.py	ProductCatalog: varunamn per EAN	motorn
voice.py	GuidanceVoice (handling till svenska) och Speaker (audio_manager eller utskrift)	beslutsreglerna
link.py	TcpJsonLink: händelser ut, kommandot say in, bara localhost	vad händelserna betyder
session.py	GuidanceSession: det enda skannern anropar (on_frame)	
replay.py	Spelar upp motorn mot gamla loggar	kameran

Delarna känner varandra via små gränssnitt (Speaker, ProductCatalog, EventPublisher), som kan bytas ut: en annan röst, butikens riktiga sortiment, en annan transport. Skannerns huvudloop fick ett enda tillägg.

3. Länken och C#

Beslutet 3/11 är ”mellanvägen”: **Python äger kameran, avkodningen, filtren och allt som rör säkerhet**, och applikationslogiken kan senare flyttas till C# eftersom konsulten som hjälper Elias kan C# bäst. Python-delen skickar JSON-rader över localhost (--lank PORT):

- **Ut:** varje läsning (loggraden oförändrad), vägledningsbeslut och ett hello med protokollversion.
- **In:** bara kommandot say. Texten läses upp på **nivå 3** i ljudkön, alltså aldrig före akutlarm.
- En klient som inte hänger med kopplas bort efter 0,2 s, så kameraloopen blockeras aldrig. Andra adresser än localhost vägras i koden.
- Pythons motor, tests/test_engine.py (varje regel ett testfall) och replay är specifikationen vid en portering.

Avkodarens gränser är uppmätta med zxing-cpp. En annan avkodare (till exempel ZXing.Net) ger andra gränser, så det uppmätta ska inte skrivas om.

4. Hur det är testat

Vad	Resultat
Enhetstester (unittest, ingen kamera)	99 tester: motorn (varje regel), modellen, rösten, länken över riktiga sockets, sessionen, uppspelningen
Hela skannern	barcode_scanner.main() körd mot en låtsaskamera med äkta streckkoder: bekräftelsen kommer en gång, okänd vara sägs som okänd, flaggan --lank öppnar en port
Mutationskontroll	15 avsiktliga fel lades in i en kopia av koden, ett i taget. 14 upptäcktes direkt. Ett överlevde (bekräftad vara guidades ändå) eftersom ett test passerade av fel anledning. Testet rättades, och nu upptäcks alla 15
Kodkvalitet	pyflakes rent. Varje ny fil har en rubrik som förklarar syfte och ändringar, som resten av repot

5. Vad som inte är testat eller är osäkert

Inget är körd på Pi:n, och ingen har hört rösten. Trösklarna är startvärden. Särskilt conflict_radius (0,10) är en gissning: för liten missar konflikter när varan rör sig, för stor kan två grannvaror på en hylla räknas som en konflikt. Hyllor är inte mätta.

- **Riktningskonventionen är en öppen fråga** till Elias: --sikte huvud (vrid huvudet, standard) eller hand (flytta varan). Välj en.
- **Rösten kan bara guida när kameran ser koden,** även som tunn läsning. Utanför bild, eller för suddig för att upptäckas, säger den inget. Löses med fokusområde (B2).
- **Mitten av en tunn läsning** är mitten av remsan som avkodades, inte hela koden, så riktningen kan vara något förskjuten.
- **Ingen pip.** Namnet räcker som bekräftelse. En pip kräver ljudstöd som inte finns i audio_manager.
- **Ljudkön kan spela gamla besked** upp till 4 s efter att de köats (nivå 3). Motorns takt gör det osannolikt.
- **Bästa fall.** Alla resultat gäller kameran rätt riktad. Om det fungerar för någon som inte ser visar bara ett test med ögonbindel.

6. Nästa steg

#	Steg	Varför
1	python3 -m voice_guidance.replay logs/scan_*.jsonl --karta kartor/produkter.geojson på Pi:n	Visar vad motorn hade sagt under alla gamla körningar, och hur ofta den hade sagt FEL vara. Ska vara 0
2	tts_engine.py --prewarm (utan kameran), sedan --vagledning text, sedan tal	Hör om beskeden stämmer med det som händer
3	Justera trösklarna och välj riktningskonvention	Startvärden är gissningar
4	Test med ögonbindel	Den enda sanna testen
5	Fokuslås (B1), fokusområde (B2), pip, burk och blank påse, USB3-test	Resten av planen